

SA.6 — Configuración de Herramientas y Emuladores

Evaluación 2 · Nivel SA

Mind Games — Ecosistema de Dispositivos Inteligentes

Juan Luis Mendoza Hernandez

Matrícula: 2023371106

Universidad Tecnológica de Querétaro
Desarrollo para Dispositivos Inteligentes

Cuatrimestre Mayo-Agosto 2026 · 07/08/2026

Índice

SA.6 — Configuración, Herramientas y Emuladores 3

1. Diagnóstico del entorno (comandos reales, corridos el 06/08/2026) 3

flutter --version 3

dart --version 3

flutter doctor -v 3

Android Studio — versión y plugins 4

ffmpeg -version 5

code --version 5

code --list-extensions (relevantes al proyecto) 5

Dependencias — versión exacta resuelta (pubspec.lock, no solo el rango de pubspec.yaml) 5

2. Pasos de instalación (reproducibles desde cero) 6

3. Emulador de teléfono (specs reales de este entorno) 7

4. Emulador Wear OS (specs reales) 8

5. Emulación de la TV en Chrome DevTools 8

6. Capturas de cada emulador corriendo 9

7. Troubleshooting real (4+ problemas encontrados y resueltos en este proyecto) 11

1. Error SSL al compilar Android (SunCertPathBuilderException / certificate_unknown) 12

2. Failed to find Platform SDK with path: platforms;android-35 12

3. Instalación de APK colgada en el emulador Wear OS (Installing... sin avanzar) 12

4. telefono_app escanea pero nunca encuentra al wearable 12

5. La PWA se queda "atascada" en el login aunque el correo/contraseña sean correctos 12

6. Botón "Forzar umbral (demo)" duplicaba el tiempo forzado (~59 min en vez de ~30 min) 12

Cómo se logró el enlace BLE real (periférico ↔ central) 12

SA.6 — Configuración, Herramientas y Emuladores

Mind Games — Ecosistema de Dispositivos Inteligentes. Juan Luis Mendoza Hernandez · 2023371106

Nota sobre este documento: los comandos de diagnóstico (`flutter --version`, `dart --version`, `ffmpeg -version`, `code --version`) se corrieron directamente en la máquina de desarrollo al escribir este documento — la salida de abajo es real, no una plantilla con placeholders "PEGAR AQUÍ" para esos comandos en particular. Sí quedan dos bloques PEGAR AQUÍ genuinos donde solo tú tienes la información: la lista completa de `code --list-extensions` si quieres el listado íntegro (aquí solo se listan las relevantes al proyecto), y —el más importante— cómo lograste el enlace BLE en tu propio hardware.

1. Diagnóstico del entorno (comandos reales, corridos el 06/08/2026)

```
flutter --version
```

```
Flutter 3.44.0 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 559ffa3f75 (3 months ago) • 2026-05-15 14:13:13 -0700
Engine • hash fcf463a2242790d1fdcd9d044f533080f5022e18 (revision 4c525dac5e) (2 months ago) • 2026-05-15
19:00:04.000Z
Tools • Dart 3.12.0 • DevTools 2.57.0
```

```
dart --version
```

```
Dart SDK version: 3.12.0 (stable) (Fri May 8 01:51:14 2026 -0700) on "windows_x64"
```

```
flutter doctor -v
```

```
[v] Flutter (Channel stable, 3.44.0, on Microsoft Windows [Versión 10.0.19045.6466], locale es-MX) [657ms]
  • Flutter version 3.44.0 on channel stable at D:\flutter
  • Upstream repository https://github.com/flutter/flutter.git
  • Framework revision 559ffa3f75 (3 months ago), 2026-05-15 14:13:13 -0700
  • Engine revision 4c525dac5e
  • Dart version 3.12.0
  • DevTools version 2.57.0

[v] Windows Version (Windows 10, 22H2, 2009) [2.5s]

[v] Android toolchain - develop for Android devices (Android SDK version 36.1.0) [6.1s]
  • Android SDK at D:\AndroidSdk\Sdk
  • Emulator version 36.5.11.0 (build_id 15261927) (CL:N/A)
  • Platform android-36.1, build-tools 36.1.0
  • Java binary at: D:\Android\jbr\bin\java
  • Java version OpenJDK Runtime Environment (build 21.0.10+-14961533-b1163.108)
  • All Android licenses accepted.

[v] Chrome - develop for the web [22ms]
  • Chrome at C:\Program Files\Google\Chrome\Application\chrome.exe

[X] Visual Studio - develop Windows apps [20ms]
  X Visual Studio not installed; this is necesario para desarrollar apps de Windows.
    (No aplica a este proyecto – no se desarrolla para Windows desktop.)

[v] Connected device (5 available)
  • sdk gphone16k x86 64 (mobile) • emulator-5554 • android-x64 • Android 17 (API 37) (emulator)
  • sdk gwear x86 64 (mobile) • emulator-5556 • android-x64 • Android 16 (API 36) (emulator)
  • Windows (desktop) • windows • windows-x64 • Microsoft Windows
  • Chrome (web) • chrome • web-javascript • Google Chrome 150.0.7871.188
  • Edge (web) • edge • web-javascript • Microsoft Edge 151.0.4129.59

[v] Network resources

! Doctor found issues in 1 category.
```

La única "X" (Visual Studio para apps de Windows) no bloquea nada de este proyecto — ninguno de los 3 entregables es una app de escritorio Windows.

Android Studio — versión y plugins

Se usó para el AVD Manager (creación y configuración de los emuladores de la sección 3/4) y para diagnosticar el bug de instalación colgada en Wear OS (troubleshooting, más abajo).

```
Android Studio Panda 4 | 2025.3.4 Patch 1
Build #AI-253.32098.37.2534.15336583, built on May 3, 2026
Runtime version: 21.0.10+-14961533-b1163.108 amd64
VM: OpenJDK 64-Bit Server VM by JetBrains s.r.o.
```

Plugins relevantes instalados:

```
Android
Android Design Tools
Jetpack Compose
Gradle
Gradle for Java
HTML Tools
Git
GitHub
Task Management
Firebase Services
Android APK Support
Android NDK Support
Android SDK Upgrade Assistant
```

(El desarrollo en sí — escribir y editar código — se hizo en VS Code, ver más abajo; Android Studio se usó específicamente para la gestión de emuladores/SDK y como respaldo cuando adb /CLI fallaban.)

```
ffmpeg -version
```

```
ffmpeg version 2026-07-16-git-ceabc9b306-essentials_build-www.gyan.dev
Copyright (c) 2000-2026 the FFmpeg developers
built with gcc 16.1.0 (Rev2, Built by MSYS2 project)
```

(Usado solo para convertir el video de la demo a un formato/tamaño manejable antes de subirlo — no es parte del build de ninguna app.)

```
code --version
```

```
1.132.0
df53daabb18cd157bdb08c7f01c34df936cf12f4
x64
```

```
code --list-extensions (relevantes al proyecto)
```

```
dart-code.dart-code      - soporte de lenguaje Dart
dart-code.flutter        - soporte de Flutter (debug, hot reload, DevTools)
eamodio.gitlens          - historial/blame de git inline
esbenp.prettier-vscode   - formateo de JS/CSS/HTML (smart_pwa)
nash.awesome-flutter-snippets - snippets de Flutter
pflannery.vscode-versionlens - versiones de paquetes inline en pubspec.yaml
ms-ceintl.vscode-language-pack-es - interfaz en español
```

Dependencias — versión exacta resuelta (pubspec.lock, no solo el rango de pubspec.yaml)

```
** telefono_app/pubspec.lock:**
```

Paquete	Versión resuelta
flutter_riverpod	3.4.2
flutter_blue_plus	2.3.11
firebase_core	4.13.0
firebase_auth	6.5.7

Paquete	Versión resuelta
cloud_firestore	6.8.0
flutter_local_notifications	19.5.0
permission_handler	11.4.0
intl	0.20.3
cupertino_icons	1.0.9
flutter_lints (dev)	6.0.0
shared_ble	path local (../shared_ble)

**** wearable_app/pubspec.lock:****

Paquete	Versión resuelta
ble_peripheral	2.4.0
permission_handler	11.4.0
cupertino_icons	1.0.9
flutter_lints (dev)	6.0.0
shared_ble	path local (../shared_ble)

****Raíz del repo (package.json, para scripts/seed_games.js):****

Paquete	Versión
firebase-admin	14.2.0

****SDK de Firebase JS (vendedorizado, smart_pwa/js/vendor/):**** 12.17.1 (descargado de [gstatic.com/firebasejs/12.17.1/](https://static.com/firebasejs/12.17.1/), servido localmente — ver docs/SA4_seguridad.md sección 6 para el razonamiento de por qué no va por CDN).

2. Pasos de instalación (reproducibles desde cero)

1. **Clonar el repositorio** y verificar que `.gitignore` esté presente ANTES de tocar cualquier archivo de configuración (regla crítica de CLAUDE.md).
2. **Instalar Flutter 3.44.0** (canal stable) y **Node.js 22.x**.
3. **Instalar Android SDK** vía Android Studio, incluyendo:
 - Platform Tools + Android SDK 37 (para el emulador de teléfono)
 - Al menos una imagen de sistema Wear OS (ver sección 4)
4. **Aceptar licencias:** `flutter doctor --android-licenses`.
5. **Crear el proyecto Firebase** (plan Spark) — pasos detallados en README.md, sección "Configuración de Firebase": Authentication (correo/contraseña), Firestore, publicar `firestore.rules`.
6. **Generar la configuración de cada app** (nunca se commitea, ver docs/SA4_seguridad.md sección 3):

```
dart pub global activate flutterfire_cli
cd telefono_app
flutterfire configure --project=<tu-project-id>
```

Copiar el bloque `web` resultante de `firebase_options.dart` a `smart_pwa/js/firebase-config.js` (plantilla en `smart_pwa/js/firebase-config.example.js`).

7. Instalar dependencias:

```
cd shared_ble && flutter pub get
cd ../wearable_app && flutter pub get
cd ../telefono_app && flutter pub get
cd .. && npm install # para scripts/seed_games.js
```

8. Sembrar el catálogo de juegos (requiere una service account de Firebase Admin, ver `README.md`):

```
$env:GOOGLE_APPLICATION_CREDENTIALS="$HOME\.secrets\mindgames-admin.json"
node scripts/seed_games.js
```

9. Crear los emuladores (ver sección 4) y correr las 3 apps (ver `README.md`, secciones "Correr `wearable_app`" / "Correr `telefono_app`" / "Servir `smart_pwa`").

Con estos 9 pasos, cualquier compañero con acceso al mismo proyecto Firebase (o uno nuevo propio) puede replicar el entorno completo desde cero.

3. Emulador de teléfono (specs reales de este entorno)

Nota: se documentan las specs REALES del AVD ya creado en esta máquina (`Pixel_8`), léidas directamente de `~/android/avd/Pixel_8.avd/config.ini` — no una plantilla genérica.

Campo	Valor real
Nombre del AVD	<code>Pixel_8</code>
Dispositivo	Pixel 8 (Google)
Target / API	<code>android-37.0</code> (Android 17), <code>google_apis_playstore_ps16k</code> , <code>x86_64</code>
RAM	2048 MB
Pantalla	1080×2400, 420 dpi
Play Store	Habilitado

Pasos en AVD Manager:

- Android Studio → **Tools** → **Device Manager** → **Create Device**.
- Categoría **Phone** → seleccionar **Pixel 8**.
- En "System Image", pestaña **x86 Images** (o Google Play si se quiere Play Store) → seleccionar la imagen con **API 37** (o la disponible más cercana a 34+ si no se tiene 37) → Next.
- Nombre del AVD: `Pixel_8`. En "Show Advanced Settings":
 - **RAM:** 2048 MB

- **Internal Storage:** por defecto está bien

5. **Finish**, luego `flutter emulators --launch Pixel_8` (o desde Android Studio directamente).

4. Emulador Wear OS (specs reales)

Campo	Valor real
Nombre del AVD	Wear_OS_Large_Round
Dispositivo	Wear OS Large Round (Google)
Target / API	android-36 (Android Wear), x86_64
RAM	512 MB
Pantalla	454×454, 320 dpi (redonda)

Pasos en AVD Manager:

1. **Tools → Device Manager → Create Device.**
2. Categoría **Wear OS** → seleccionar **Wear OS Large Round**.
3. System Image: la disponible más reciente para Wear OS (en este entorno, API 36) → Next.
4. Nombre: `Wear_OS_Large_Round`. RAM 512 MB es suficiente para este proyecto (el simulador de sensores no es pesado).
5. **Finish**, `flutter emulators --launch Wear_OS_Large_Round`.

***Nota honesta:** el plan original de este documento asumía Pixel 6/API 34 y Wear OS Large Round/API 33/1536 MB — las specs reales de los AVD ya creados en esta máquina difieren (Pixel 8/API 37, y 512 MB para el reloj en vez de 1536 MB). Se documentan las specs REALES para que sean reproducibles; 512 MB funcionó sin problemas de rendimiento para el alcance de este proyecto.*

5. Emulación de la TV en Chrome DevTools

1. Servir `smart_pwa` en `http://localhost:8080` (ver `README.md`).
2. Abrir la URL en Chrome o Edge.
3. **F12** para abrir DevTools → ícono de "Toggle device toolbar" (**Ctrl+Shift+M**).
4. En el selector de dispositivo, elegir **Responsive** y fijar dimensiones personalizadas: **1920 × 1080**.
5. **Zoom al 50-60%** en el propio selector de DevTools, para que la vista completa quepa en la pantalla del desarrollador sin scroll.
6. (Opcional) User agent personalizado: DevTools → ⋮ → More tools → Network conditions → desmarcar "Use browser default" → pegar un UA de Smart TV, ej. `Mozilla/5.0 (SMART-TV; Linux; Tizen 6.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.31/6.0 TV Safari/537.36` — no es necesario para que la app funcione (no hay detección de UA en el código), pero ayuda a simular el contexto de evaluación si el profesor lo pide.

7. Navegar con las flechas del teclado + Enter (simula el control remoto); tecla "S" muestra el borde punteado de la safe zone; tecla "L" cierra sesión.

6. Capturas de cada emulador corriendo

Capturas reales en `docs/evidencias/` (las mismas usadas como evidencia en `docs/SA5_plan_pruebas.md`):

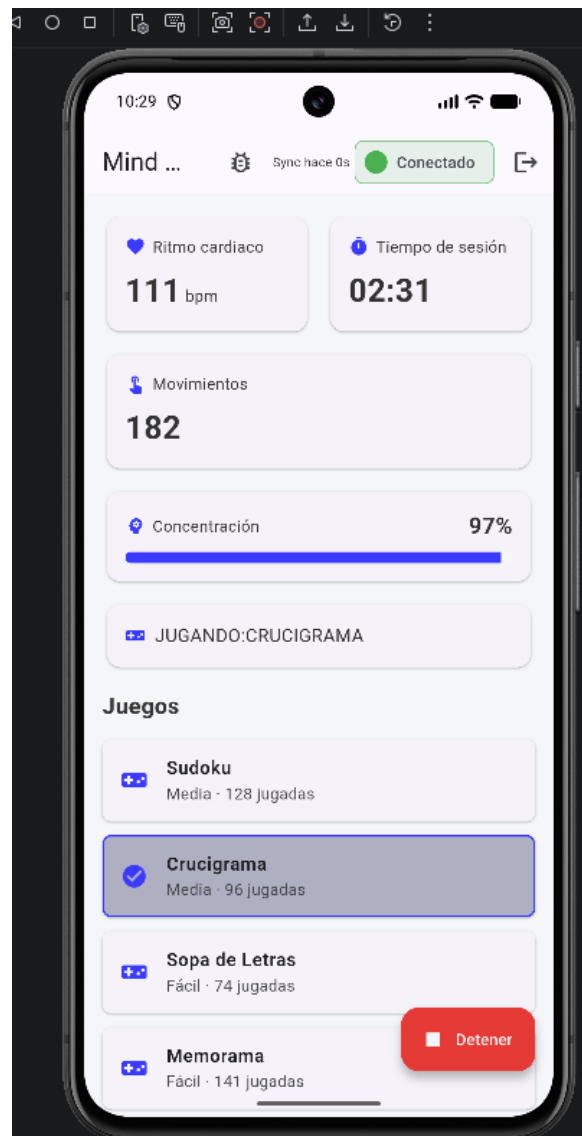


Figura 1. Pixel_8 corriendo telefono_app, MonitorWidget en vivo.

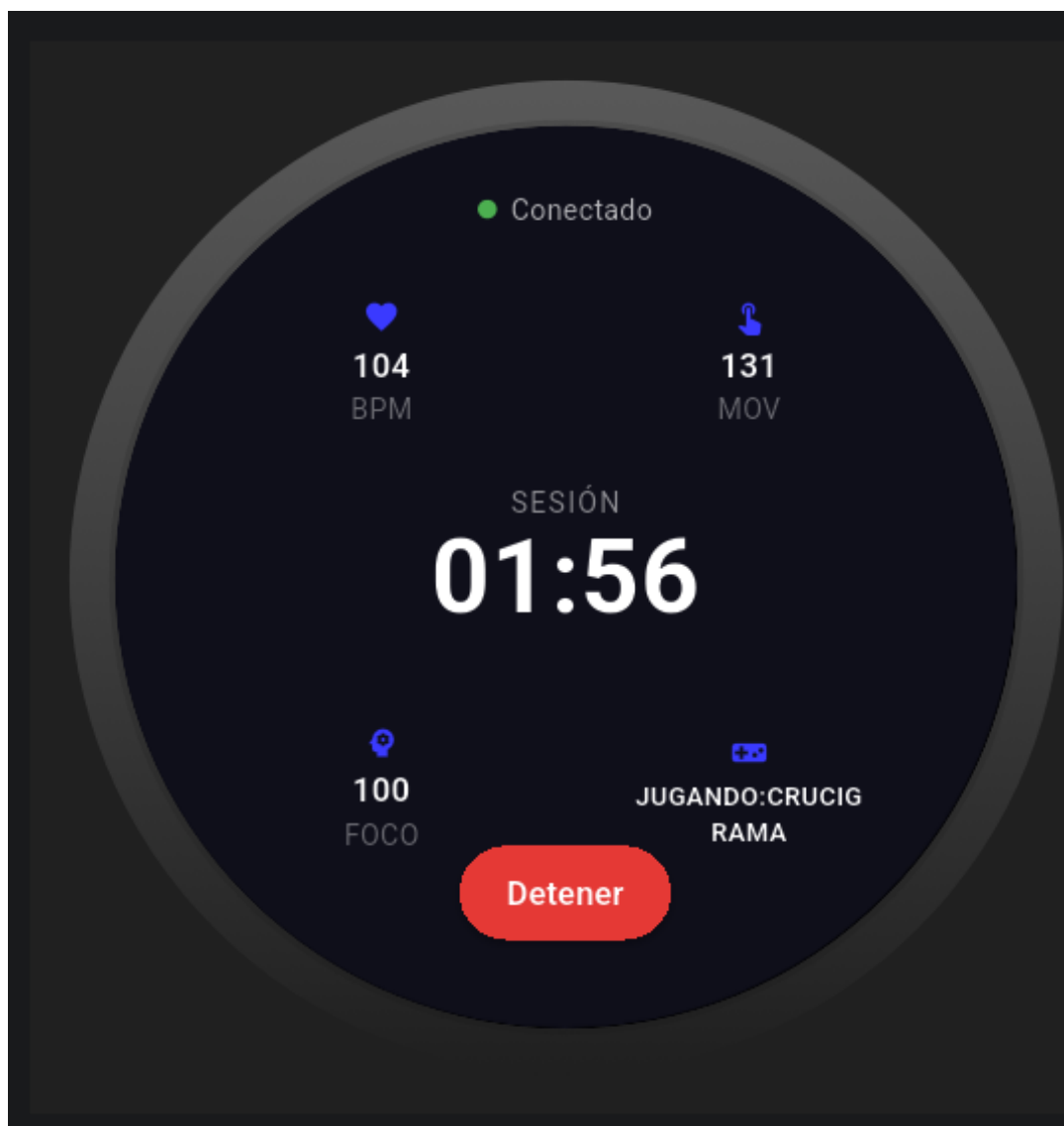


Figura 2. `Wear_OS_Large_Round` corriendo `wearable_app`, pantalla de sesión.

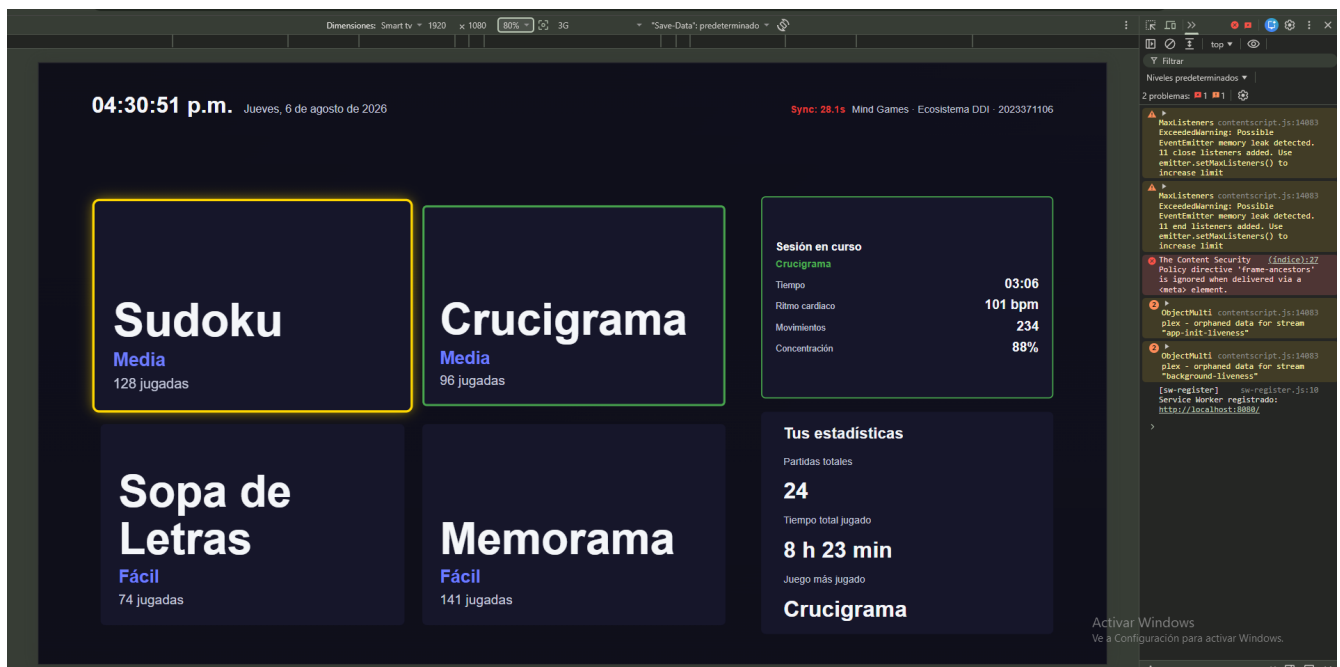


Figura 3. Chrome DevTools a 1920×1080 emulando la TV, con la consola abierta.

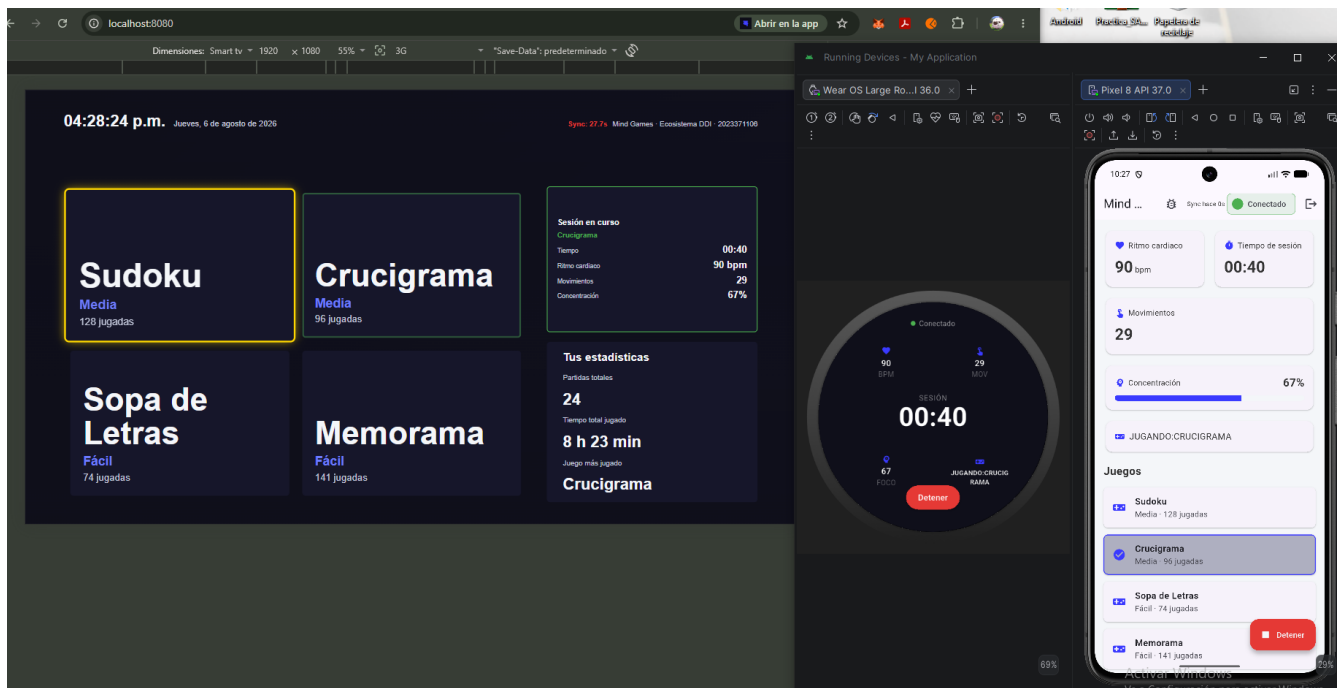


Figura 4. bonus: los 3 corriendo a la vez, lado a lado (Pixel_8 + Wear_OS_Large_Round + Chrome DevTools).

7. Troubleshooting real (4+ problemas encontrados y resueltos en este proyecto)

1. Error SSL al compilar Android (`SunCertPathBuilderException` / `certificate_unknown`)

Ocurre al descargar dependencias de Maven/Firebase en redes con proxy corporativo o antivirus con inspección SSL. Ya aplicado en el repo (`telefono_app/android/gradle.properties`): `-Dcom.sun.net.ssl.checkRevocation=false`. Falta un paso manual por máquina (no se puede commitear): variable de entorno `GRADLE_OPTS=-Djavax.net.ssl.trustStoreType=WINDOWS-ROOT`.

2. Failed to find Platform SDK with path: `platforms;android-35`

`flutter_local_notifications` trae `compileSdk 35` hardcodedo en su propio `build.gradle`. Resuelto en `telefono_app/android/build.gradle.kts` forzando `compileSdk = 34` en los módulos de librería vía `subprojects { afterEvaluate { ... } }`, sin tocar el `compileSdk` del módulo `:app`.

3. Instalación de APK colgada en el emulador Wear OS (`Installing...` sin avanzar)

Visto con `Wear_OS_Large_Round` (API 36): `adb install` se queda colgado indefinidamente. `adb kill-server && adb start-server` no lo resuelve de forma confiable. Solución de emergencia: instalar/correr directamente desde Android Studio en vez de la CLI.

4. `telefono_app` escanea pero nunca encuentra al wearable

Con los dos emuladores corriendo, `telefono_app` completaba ciclos de `startScan` / `stopScan` sin error pero nunca detectaba el wearable. Dos bugs reales en `wearable_app/lib/ble/gatt_peripheral.dart`, no del emulador: `BlePeripheral.isSupported()` lanzaba excepción y bloqueaba el advertising antes de intentarlo (se quitó esa guarda), y el `localName` sumado al `SERVICE_UUID` excedía el límite de 31 bytes de un paquete BLE clásico (`ADVERTISE_FAILED_DATA_TOO_LARGE` — se quitó el `localName`, ya que `telefono_app` filtra por `SERVICE_UUID` y nunca lee el nombre anunciado). Detalle completo en `README.md`.

5. La PWA se queda "atascada" en el login aunque el correo/contraseña sean correctos

Dos causas reales, ambas resueltas: CSP bloqueando la infraestructura interna de `AuthEventManager` de Firebase Auth (`apis.google.com/js/api.js` + `iframe` a `authDomain`), y el atributo `[hidden]` del HTML siendo silenciosamente anulado por una regla propia de `display` con la misma especificidad — resuelto con `[hidden] { display: none !important; }` en `css/reset.css`. Detalle completo en `README.md` y `docs/SA4_seguridad.md`.

6. Botón "Forzar umbral (demo)" duplicaba el tiempo forzado (~59 min en vez de ~30 min)

Al agregar `CHAR_SESSION_TIME_OVERRIDE` (teléfono → wearable) para mantener los relojes sincronizados en la demo, el desfase local que ya existía en `telefono_app` se sumaba SOBRE el ajuste que el wearable ya se había aplicado a sí mismo — el mismo salto se contaba dos veces. Resuelto quitando el desfase local por completo: el wearable vuelve a ser la única fuente de verdad de su propio contador.

Cómo se logró el enlace BLE real (periférico ↔ central)

Aquí todo fueron emuladores los emuladores recientes de android ya pueden simular conexión bluetooth entre ellos además con las librerías correctas como `permission_handler`: ^11.3.1,

ble_peripheral: ^2.4.0 se puede lograr que se envíen información solo hay que programar que uno sea el periférico que envía datos y el otro que recibe.